IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA (DEEMED TO BE UNIVERSITY)

Yenepoya Institute of Arts, Science, Commerce and Management

Project – High Level design

on

Natural Language interface for EdTech infrastructure

Student Name: Fathima Sherin

Industry Mentor:

Guided by: MS Manjula

Table of Contents

1. Introduction 2
1.1. Scope of the document 2
1.2. Intended Audience 2
1.3. System overview 2
2. System Design 3
2.1. Application Design 3
2.2. Process Flow 4
2.3. Information Flow 5
2.4. Components Design 6
2.5. Key Design Considerations 6
2.6. API Catalogue 7
3. Data Design 7
3.1. Data Model 7
3.2. Data Access Mechanism 7
3.3. Data Retention Policies 8
3.4 Data Migration 8
4. Interfaces 8
4.1. Web Interface 8
4.2. Dashboard interface 9
5. State and Session Management 9
6. Caching 9
7. Non-Functional Requirements 10
7.1. Security Aspects 10
7.2. Performance Aspects 10
8. Reference 11

1. Introduction

Edu Bot is a simple chatbot built using Python and Flask. It helps students get quick answers to common questions about courses, fees, enrollment, schedules, and certificates. The chatbot works by matching keywords in the student's message and returning a pre-defined reply. It does not use any AI or machine learning. The project also has a small dashboard page that shows basic statistics about how the chatbot is being used.

1.1. Scope of the document

This document gives an overview of how Edu Bot is designed. It covers the main parts and how the different components work together. It does not go into detailed code-level explanations. That is covered in Low Level Design document.

1.2. Intended Audience

This document is written for:

- Internal guide and project evaluators
- Developers who want to understand or extend the system
- Students who want to learn how the project is structured

1.3. System overview

Edu Bot is a web-based chatbot application. It has three main parts. The first part is the web page where the student types their question. The second part is the Python Flask server that receives the question and processes it. The third part is keyword matching logic that finds the correct answer and sends it back to the student.

The project also includes a dashboard page that shows how many questions were asked, what topics were asked about, and some basic statistics from Udemy courses dataset.

2. System Design

This section explains how Edu Bot is designed and how it works from the time a student types a question to the time they get an answer.

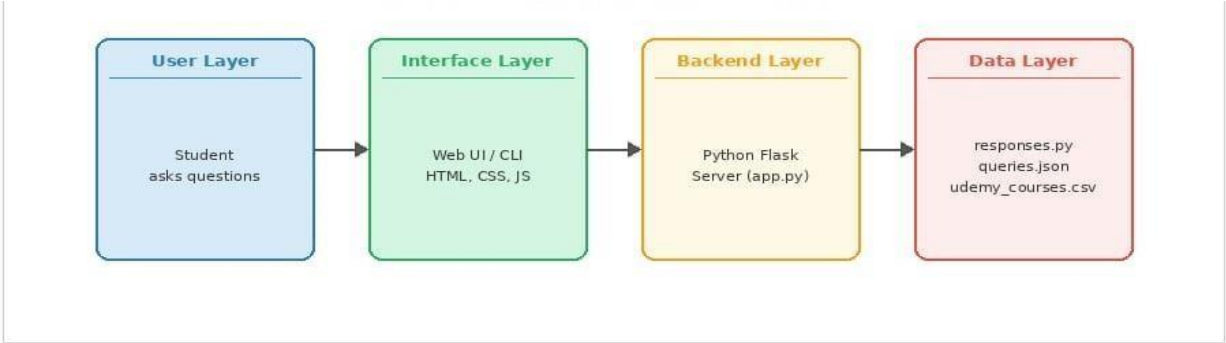
2.1. Application Design

Edu Bot is built using a simple three-layer structure.

- The first layer is the web page. This is what the student sees and uses to type their question.
- Second layer is the Flask server. This receives the question from the webpage and passes it to the chatbot logic.
- Third layer is the chatbot logic. This checks the question for keywords and returns the correct answer.

There is also a separate analytics layer that handles the dashboard page and reads data from Udemy courses file.

High Level architecture:



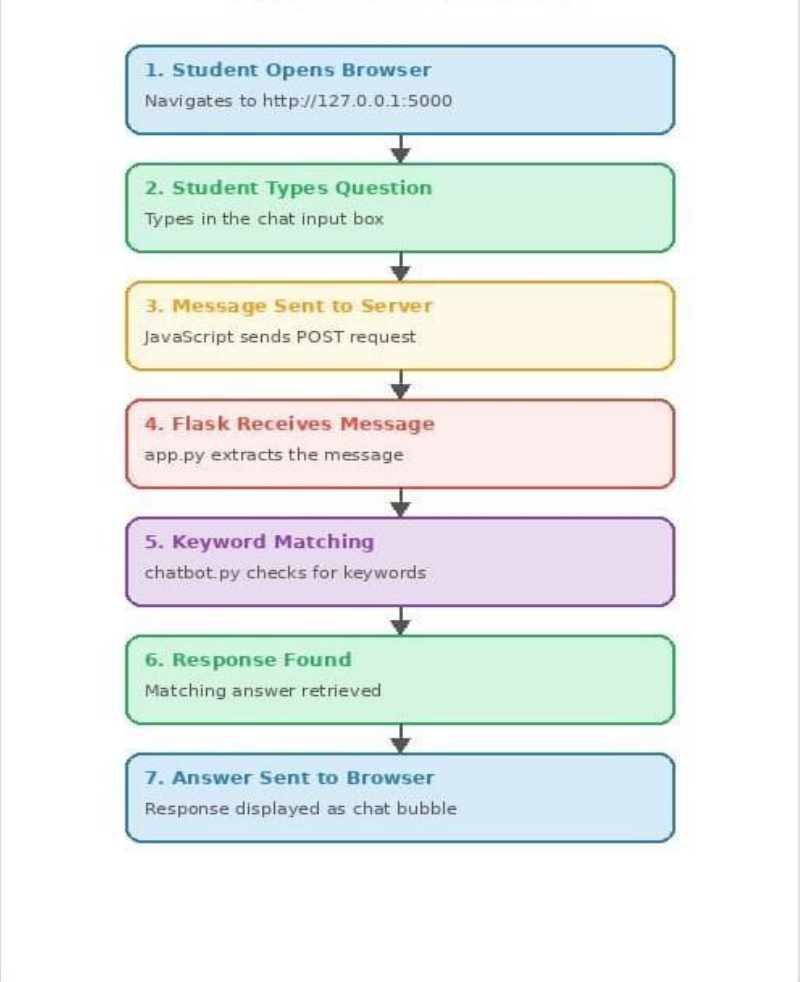
2.2. Process Flow

The following steps show how Edu Bot handles a student's question:

1. The student opens the browser and goes to the Edu Bot page.

2. The student types a question in the chat box and clicks send.
3. The question is sent to the Flask server.
4. The server passes the question to the chatbot logic.
5. The chatbot checks for matching keywords in the question.
6. The matching answer is sent back to the browser.
7. The answer is displayed to the student as a chat message.

Process Flow Diagram:

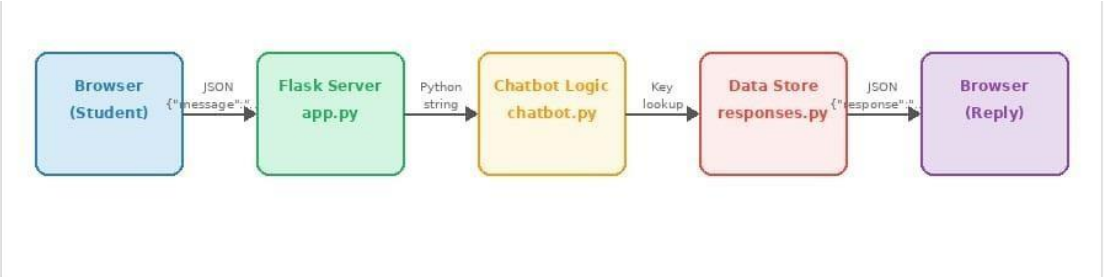


2.3. Information Flow

This section shows how information moves through the system.

- The student types a message in the browser and clicks send.
- The browser sends the message to the Flask server in JSON format.
- The Flask server passes the message to the chatbot logic.
- The chatbot logic checks the message and finds the correct response.
- The response is sent back to the browser in JSON format.
- The browser displays the response as a chat message.

Information Flow:



2.4. Components Design

Edu Bot is made up of following main components:

- **App.py** - This is the main Flask server file. It handles all the routes and connects the web page to the chatbot logic.
- **Chatbot.py** - This file contains the keyword matching logic. It takes the student's message and finds the correct response.
- **Response.py** - This file stores all the questions and answers in a python dictionary.
- **Index.html** - This is the main web page that the student uses to chat with Edu Bot.
- **Style.css** - This file controls how the web page looks.
- Script.js - This file handles the chat behaviour in the browser.
- **Dashboard.html** - This is the analytics page that shows chatbot usage statistics.
- **Queries.json** - This file saves every question asked by students.
- **Udemy_courses.csv** - This is the Kaggle dataset used for the dashboard charts.

2.5. Key Design Considerations

The following design decisions were made while building Edu Bot:

- Keyword matching was used instead of AI because it is simple, fast, and easy to understand. It also does not need any external tools or training data.
- Flask was chosen as the backend because it is lightweight and easy to set up for small project like this.
- Responses.py is kept separate from chatbot.py so that answers can be updated easily without changing the logic.
- A separate dashboard page was created so that the chat interface stays clean and simple for students.
- No database is used because the data is small enough to store in Python files and JSON log file.

2.6. API Catalogue

Edu Bot has the following API end points:

Endpoint	Method	Description
/	GET	Opens the main Edu Bot chat page
/chat	POST	Receives student message and returns bot reply
/dashboard	GET	Opens the analytics dashboard page

3. Data Design

This section explains how data is stored and used in Edu Bot.

3.1. Data Model

Edu Bot does not use a database. Instead, it uses three simple data sources:

- Responses.py - A Python dictionary that stores keyword and answer pairs. For example, if the student types “fee”, the chatbot looks up “fee” in the dictionary and returns the answer.

- Queries.json - A JSON file that saves every question asked by the student along with the date, time, and category of the question.
- Udemy_courses.csv - A dataset downloaded from Kaggle that contains information about 3,678 Udemy courses. This is used only for the dashboard charts.

3.2. Data Access Mechanism

Edu Bot access data in three simple ways:

- Dictionary lookup – When a student sends a message, the chatbot checks if the message contains any keyword from the responses.py dictionary and returns the matching answer.
- JSON file read and write – Every time a student asks a question, the details are saved into queries.json. The dashboard reads this file to show usage statistics.
- CSV file read – The udemy_courses.csv file is read by the dashboard page to show course statistics and charts.

3.3. Data Retention Policies

Edu Bot does not store any personal information about students. The following points explain how data is handled:

- The queries.json file only saves the question text, date, time, and category. No student name or personal details are saved.
- The chat history on the web page is only visible until the page is refreshed. After that is gone.
- The udemy_courses.csv file is never changed by the system. It is only read.

3.4 Data Migration

At this stage, data migration is not needed because Edu Bot uses simple files to store data. There is no database to migrate.

If the project is upgraded in the future, the data from responses.py and queries.json can be moved to a proper database like SQLite or MySQL.

4. Interfaces

Edu Bot has two interfaces that students can use.

4.1. Web Interface

The web interface is the main chat page. It opens in a browser at http://127.0.0.1:5000. It has the following parts:

- A navigation bar at the top with links and a Dashboard button
- A course cards panel on the left side
- A chat panel on the right side where students type their questions
- A footer at the bottom showing the student name and technology used

4.2. Dashboard interface

The dashboard page opens at http://127.0.0.1:5000/dashboard. It is meant for checking how the chatbot is being used. It shows the following:

- Total number of questions asked
- A chart showing which topics were asked about the most
- A chart showing daily usage over time
- A table showing the most common keywords used by students
- Basic statistics from Udemy courses dataset such as total courses, paid vs free count, and average price

5. State and Session Management

Edu Bot does not store any session information between requests. Each time a student sends a message, it is treated as a completely new request.

The following points explain how state is managed:

- The chat history shown on the web page is stored only in browser. It disappears when the page is refreshed.
- A question counter is kept in memory while the server running. It resets when the server is restarted.
- The queries.json file keeps a permanent record of all questions asked. This is used by the dashboard.

- There is no login or user account system in the current version.

6. Caching

Edu Bot does not use any special caching system. The following points explain how the system handles this:

- All the chatbot responses are already stored in memory when the server starts. So every keyword lookup is very faster and does not need caching.
- The browser may automatically cache static file like CSS and JavaScript to load the page faster.
- The Udemy dataset is read from the CSV file every time the dashboard page is opened. In a future version this could be cached to make the dashboard load faster.

7. Non-Functional Requirements

This section covers the security and performance aspects of Edu Bot.

7.1. Security Aspects

The following security points apply to Edu Bot:

- No personal data or login details are collected from students.
- The chatbot only stores the question text, time, date, and category in queries.json.
- The system runs locally on the student's computer and is not exposed to the internet.
- No sensitive information is stored anywhere in the project.

7.2. Performance Aspects

The following points explain the performance of Edu Bot:

- The chatbot responds very quickly because all answers are stored in memory and no database is used.
- A small delay of 0.5 to 0.15 seconds is added on the web page to make the chat feel more natural.
- The system is designed for single user use and works well for demo and academic purposes.

- The dashboard loads the Udemy dataset from a CSV file each time it is opened. This is acceptable for the current scale of the project.

8. Reference

9. README.md
10. App.py
11. Chatbot.py
12. Responses.py
13. Queries.py
14. Udemy_courses.csv
15. Templates/index.html
16. Templates/dashboard.html
17. Static/style.css
18. Static/script.js